STRATA: One Geometry-Free Persistence-and-Periodicity Engine Driving Selectable 2D/3D Lifelong LiDAR Mapping in ROS 2

Jungmo Kang
Independent Researcher
kangjmo91@gmail.com
https://github.com/kjungmo

Abstract—A robot that maps the same building for weeks cannot bake every LiDAR hit permanently into one grid: parked carts, passing people, and doors that open on a schedule all burn in, and the map slowly ossifies into a record of everything that was ever seen rather than of what is actually there now. Lifelong mapping must instead hold three properties in tension at once — durability (keep the walls), plasticity (forget the movers), and periodicity (treat a cyclic door as signal, not noise). We present STRATA, a small, geometry-free persistence-and-periodicity engine that resolves this tension in one shared per-cell state machine and drives two pluggable geometry backends — a fixed-array 2D occupancy grid and an unbounded 3D voxel-hash — behind a single interface selected by one runtime string parameter. The only per-backend code is point-to-id mapping and the free-space ray walk; the classifier that graduates, demotes, prunes, and periodicity-labels every cell is shared by composition, not duplicated per dimension. The engine is a plain C++17 + Eigen library with no ROS, DDS, or PCL dependency, exercised by 22 behavior-level tests, so its scientific logic builds and runs deterministically in a second-scale CI loop. A seeded synthetic characterization suite reports that both backends graduate a static wall by the third window and hold recall 1.0 thereafter (their only divergence, a clutter-induced static-precision gap of 0.9253 vs. 0.9758 under 100 movers per window, is geometric, not algorithmic); that 50%-duty periodic doors are detected at true-positive rate 2/3 and false-positive rate 1/5; that removing hysteresis inflates flicker from 54 to 574 toggles on identical replayed noise (at fixed decay $\lambda = 0.90$); and that the unified engine costs a flat $\sim$56 B per cell with no per-dimension penalty. Evaluation is synthetic and the tool performs no SLAM; STRATA is the persistence core meant to sit beneath an external localizer, released open-source as a ROS 2 package.

Index Terms—lifelong mapping, occupancy grids, voxel mapping, persistence filtering, FreMEn periodicity, ROS 2, open-source robotics software

I. Introduction

An occupancy grid built the textbook way accumulates evidence and never lets go of it. For a robot that drives a route once, that is fine. For one that maps the same warehouse or hospital corridor for weeks, it is a slow failure: every hit is folded into the same static belief, so a cart parked for an afternoon, a person walking past, and a load-bearing wall all leave the same kind of mark. Over enough passes the map stops describing the building and starts describing the union of everything the sensor ever touched. Suppressing that failure is the whole problem of lifelong mapping, and it forces three requirements that pull in different directions: the map must be durable enough to keep structure that is genuinely permanent, plastic enough to forget structure that has moved on, and must do both without a single global forgetting rate that is simultaneously too slow to erase movers and too fast to trust a wall. This is the stability–plasticity dilemma that Biber and Duckett framed for dynamic maps [1]: represent the environment at more than one timescale at once, or lose either the walls or the ability to adapt.

A third requirement complicates the split further. Some structure is neither permanent nor transient but cyclic — a door open every morning and shut every night, a shutter that tracks business hours. Krajník et al. show that such a cell's occupancy is a periodic temporal signal, and that a flat decay law erases exactly the regularity that makes it predictable, discarding information a robot could have exploited [2], [3]. A lifelong map therefore needs three coexisting timescales, not two: a fast-fading occupancy signal, a slowly-graduated durable class, and a separately-timed periodic class that a decay term alone would flatten into noise.

The systems that come closest to solving this are heavier than the problem needs to be. ELite [4] and LT-mapper [5] — the closest published twins in intent — resolve durability versus plasticity, but as parts of full lifelong-SLAM pipelines, coupled to multi-session point-cloud registration, place recognition, and alignment. RTAB-Map [6] is the closest engineering precedent for offering selectable 2D and 3D geometry under one framework, but it too is a complete SLAM system with loop closure and memory tiering. Each carries the persistence-and-classification logic a robot needs, but welded inside a much larger apparatus and to a specific map geometry. What is missing from the surveyed prior art is the small piece by itself: a standalone, SLAM-free, dependency-light engine

that does only the graduation, forgetting, and periodicity classification, that runs under either 2D or 3D geometry, and that a practitioner can drop beneath an external localizer or an existing SLAM front end.

STRATA is that piece. The name is literal: a cell in the map carries temporal strata — Static, Periodic, and Transient layers over an Unknown baseline — coexisting inside one per-cell state machine, read and written by log-odds occupancy, survival decay, Schmitt-trigger hysteresis, and an incremental Fourier periodicity test. That state machine is geometry-free. Both shipped backends translate world points into int64 cell keys and walk a free-space ray to clear it; everything downstream of the key — accumulation, graduation, demotion, pruning, periodicity labeling — happens once, in the shared engine. The thesis is deliberately narrow: the only per-backend code is point-to-id mapping and the free-space ray walk; the classifier is shared by composition, not duplicated per dimension (Fig. 1). Selecting 2D versus 3D is a single runtime string parameter resolved at construction, not a plugin-loading mechanism, and the engine itself (strata_core) is a plain C++17 + Eigen library with no rclcpp, tf2, PCL, or DDS dependency, so its scientific logic builds and unit-tests without a ROS install.

We are explicit about scope up front (§VI consolidates the non-claims). STRATA is not SLAM: it estimates no pose, closes no loops, and aligns no sessions, consuming instead an external map → sensor transform. Its evaluation is synthetic only — every number below comes from deterministic seeded harnesses and unit tests, not a field deployment, in contrast to the multi-day validations of ELite, LT-mapper, and Berrio et al. [7]. And it models single-session temporal persistence, not multi-session remapping: a graduated cell simply is the current static map, with no session versioning. These are the boundaries of the tool, stated as scope rather than buried as gaps.

Contributions. This paper makes five, each backed only by the shipped code and the synthetic suite:

1) One geometry-free engine, two geometries, one switch (by construction). A single int64-keyed persistence-and-periodicity engine (LayeredMap + PeriodicityModel) drives both a fixed-array 2D occupancy grid and an unbounded 3D voxel-hash behind one MapBackend interface, chosen by a single runtime string parameter, with no state-machine logic duplicated across dimensions (Fig. 1; §III–IV).
2) A ROS-free, deterministically testable core, shown backend-equivalent (design goal + measured). The engine builds and unit-tests with a plain C++17 + Eigen toolchain and is exercised by 22 behavior-level tests; both backends graduate the static wall by window 3 and hold recall 1.0 through window 39, differing in static-layer quality only by a clutter-induced static-precision gap (0.9253 grid2d vs. 0.9758 voxel3d at 100 movers per window) that traces to 2D z-collapse geometry, not to the classifier; a separate per-integrate() cost gap between the backends is characterized in contribution 5 (§III, §V).
3) A specific minimal combination not previously packaged standalone. Persistence-Filter survival decay [8], Removert-motivated Schmitt hysteresis [9], ReFusion-style negative-evidence ray clearing [10], and a parallel FreMEn-lite periodicity classifier [2], emitting a four-class label from one dependency-light, SLAM-free module — a combination absent, in this form, from the surveyed prior art (§II, §IV).
4) A mechanistic characterization of the shared classifier (measured). 50%-duty periodicity is detected cleanly (dominant-harmonic amplitude 0.653/0.707, well above the 0.3 threshold) at true-positive rate 2/3 and false-positive rate 1/5, with the single miss diagnosed as a prune/maturity race; and hysteresis is the dominant stabilizer — removing it yields 574 flicker toggles at F1 0.810 versus 54 toggles at F1 1.000 on identical replayed noise at the same decay ($\lambda = 0.90$) (§V).
5) A unified engine that adds no per-dimension cost, released as a reproducible tool (measured). Memory is a flat ∼56 B per cell and per-window cost tracks live-cell count linearly; the 5.5–14.9× per-integrate() gap between backends is entirely 3D free-space voxel proliferation (4.4–10.0× more live cells), geometry rather than the engine. The whole system ships as an open-source ROS 2 package with a documented I/O contract and a seeded harness whose figures regenerate from measured CSVs (§III, §V).

STRATA is available at github.com/kjungmo/strata as a ROS 2 Humble package under an open-source license, with a thin ROS adapter node and a reusable characterization harness; it is designed to plug beneath an external localizer such as prism_loc. The remainder of the paper positions STRATA against the prior art (§II), specifies its two-band, testable architecture (§III), gives the as-shipped engine mathematics including three honest implementation-versus- specification differences (§IV), reports the synthetic evaluation (§V), and states the limitations, boundaries, and reproducibility of the tool (§VI–§VII).

II. Related Work

We organize prior work into six themes and, for each entry, state STRATA's exact relation rather than a generic contrast.

Lifelong and persistent mapping. ELite [4] and LT-mapper [5] are STRATA's closest published twins. ELite computes a two-timescale ephemerality score per point (within-session dynamic vs. across-session transient) and drives a Lifelong/Static/Delta map triad; LT-mapper composes a live map, a meta map, and a delta map through explicit LT-SLAM alignment, LT-removert removal, and LT-map graduation stages. Both are full

lifelong-SLAM pipelines with multi-session point-cloud registration and place recognition. STRATA implements only the graduation half of that pattern — one per-cell state machine (Static/Periodic/Transient/Unknown, §IV-E) inside a ROS-free engine, with no scan matching, no loop closure, and no cross-session alignment. STRATA is not a competing full system; it is the persistence-and-classification core that ELite's triad and LT-mapper's live-to-static pipeline wrap around, and its measured evaluation (§V) is correspondingly narrower — a synthetic, single-session characterization, not the field-scale validation ELite and LT-mapper report. Khronos [11] generalizes the same two-timescale idea to an active-window/long-term-reconstruction split with semantics and a scene graph; STRATA stays at the raw per-cell level with no object or scene-graph layer, a narrower and cheaper instance of the same principle. Biber and Duckett [1] and Meyer-Delius et al. [12] are the foundational multi-timescale and static/temporary-split ideas STRATA's window/decay/hysteresis/periodicity combination descends from. RTAB-Map [6] is the closest engineering precedent for selectable 2D/3D geometry inside one shipped framework, coupled to full SLAM and STM/WM/LTM memory management; STRATA narrows this to the mapping/persistence layer alone, behind a MapBackend interface selected by a single string parameter, consuming an externally supplied pose rather than estimating one. Berrio et al. [7] report an 18-month field deployment of the same purge/promote pattern STRATA implements as a Schmitt trigger (§IV-C) — field-scale evidence for the pattern that STRATA currently lacks for itself. Yang et al. [13] keep positive/negative version deltas over a base map so any past session can be reconstructed; STRATA does no session versioning at all — a graduated cell is simply part of the current static map, its history implicit in log-odds and observation counts, not explicitly replayable.

Persistence and forgetting. The Persistence Filter [8] recursively estimates each feature's survival probability from a Bayesian survival-time model. STRATA's per-window survival-decay multiplier λ (§IV-B) is a discretized, single-scalar simplification of that idea: one tunable multiplicative factor pulls belief toward unknown each window in place of a full survival-time posterior. The log-odds hit/miss update with clamping in Thrun et al. [14] is used verbatim as STRATA's per-window occupancy accumulation, with decay and hysteresis layered on top. Tipaldi et al. [15] and Saarinen et al. (iMac) [16] model each cell as a two-state Markov process with a recency-weighted or online-learned transition rate — the principled, per-cell-learned form of "how fast a cell should forget." survival_decay is a single global constant, a constant-rate special case of that formalism; STRATA states this as a documented simplification (§VI), not an unacknowledged gap.

Periodicity. FreMEn [2] is the direct counter-argument STRATA's Periodic class answers: a cell's occupancy can be a periodic temporal signal, and a flat decay erases exactly that structure. STRATA's PeriodicityModel (§IV-D) runs FreMEn's incremental-Fourier accumulation in parallel with decay and graduation, so an oscillating cell is classified Periodic rather than mis-graduated to Static or mis-pruned as Transient — the third class exists specifically to answer this argument, at the cost of a bounded harmonic count and a validity gate tied to touched-window count rather than elapsed time (a documented implementation-versus-specification divergence, §IV-H). Krajník et al. [3] is the earlier, more general spectral-analysis lineage FreMEn later packaged as a mapping method; STRATA's bounded incremental-Fourier approximation is a further narrowing of that lineage, not the full spectral machinery.

Dynamic-object removal. Removert [9] conservatively removes dynamic points and then reverts wrongly removed static ones, because aggressive removal erases real structure. This is the exact justification for STRATA's hysteresis band (§IV-C): a graduated cell demotes only when its probability falls below $p_{\mathrm{dem}} < p_{\mathrm{grad}}$, i.e. sustained contradicting evidence across windows rather than a single contradicting frame — the same failure mode Removert's revert pass patches after the fact, STRATA prevents online by construction. ReFusion [10] treats a reliably observed empty voxel as evidence, not merely an absence of evidence; STRATA's ray-clearing — exact Bresenham for grid2d, sub-voxel-step sampling for voxel3d (§IV-F) — applies the same negative-information logic online, with misses accumulating as l_{miss} and eventually demoting a graduated cell once an object vacates it. ERASOR [17] removes dynamic points from an already-accumulated 3D map offline via per-bin pseudo-occupancy ratios; STRATA never batches and performs no object-level reasoning — every cell's class is a running online per-window hit/miss-and-decay state, the baseline ERASOR's batch, object-level approach is not attempting to be.

Production stacks. OctoMap [18] shares STRATA's log-odds-with-clamping core but compresses storage with an octree; voxel3d instead uses a flat int64-keyed hash for O(1) insert/lookup, trading multi-resolution compression for simplicity and unit-testability (§VI). The voxel-hashing scheme of Nießner et al. [19] is the direct data-structure this hash follows. UFOMap [20] makes "never observed" an explicit third per-voxel state; STRATA gets the same distinction implicitly from the observations counter and log-odds value rather than a dedicated state, cheaper to implement and less explicit to query. Nav2 STVL [21] decays costmap obstacle evidence for short-horizon planning using the same decay-rate mechanism family STRATA's survival_decay uses to decay mapping evidence toward unknown — same knob, different consumer. Layered Costmaps [22] composite independently maintained plugin layers per cell; STRATA's output value convention (static→100, periodic→75, transient→50, unknown→ −1) reproduces the same layered mental model as the output

of one cell-level state machine rather than as multiple composited layers. SLAM Toolbox [23] bounds compute by adding and removing pose-graph nodes while performing its own 2D SLAM; STRATA has no pose graph and no SLAM, consuming an externally supplied transform and pruning at the per-cell evidence level (§IV-E) instead — a different layer of the stack, included as the standard 2D lifelong-mapping comparison point.

Pluggable-backend architecture. ROS 2 Pluginlib [24] is the canonical swappable-backend mechanism — abstract base class, XML plugin description, runtime dlopen-based loading — and the pattern STRATA deliberately rejects, not adopts. With exactly two backends, STRATA resolves MapBackend to one of two unique_ptrs at construction from a single runtime string parameter (§III); no plugin XML, no dynamic loading, no ament_index registry. This is the right choice at two backends; pluginlib becomes the right choice only once backend count grows past what a single if/else factory reads clearly. RTAB-Map [6] is re-cited here as the closest existing shipped system offering selectable 2D/3D geometry under one framework, narrowed by STRATA to a persistence-only module with the classifier shared by composition rather than duplicated per geometry — the only per-backend code is point-to-CellId mapping and the free-space ray walk.

None of the surveyed prior art offers, as a standalone, SLAM-free, dependency-light module, the specific combination STRATA ships: survival decay, Schmitt-trigger hysteresis, negative-evidence ray clearing, and parallel FreMEn-lite periodicity classification, unified behind one geometry-free engine and selectable between two map geometries at runtime.

III. System Overview and Architecture

STRATA is delivered as two ament packages with a deliberate dependency boundary between them. strata_core is the scientific engine: pure C++17 with Eigen as its only third-party dependency, holding the entire occupancy/persistence/periodicity state machine and both geometry backends. strata is a thin ROS 2 node that wraps the engine, owning message conversion, TF lookups, publishers, subscribers, services, and file I/O. Every ROS-only dependency — rclcpp, tf2, sensor_msgs, nav_msgs, PCL — lives exclusively in strata; strata_core links none of them. This split is the reason the engine builds and unit-tests with a plain system toolchain (cmake -S strata_core -B build -DSTRATA_CORE_BUILD_TESTS=ON && ctest), with no ROS install, no colcon, no DDS discovery, and no TF buffer warm-up in the loop. The architecture is shown in Fig. 1.

A. The MapBackend interface and the geometry-free invariant

strata_core::MapBackend is a pure abstract base with four methods:

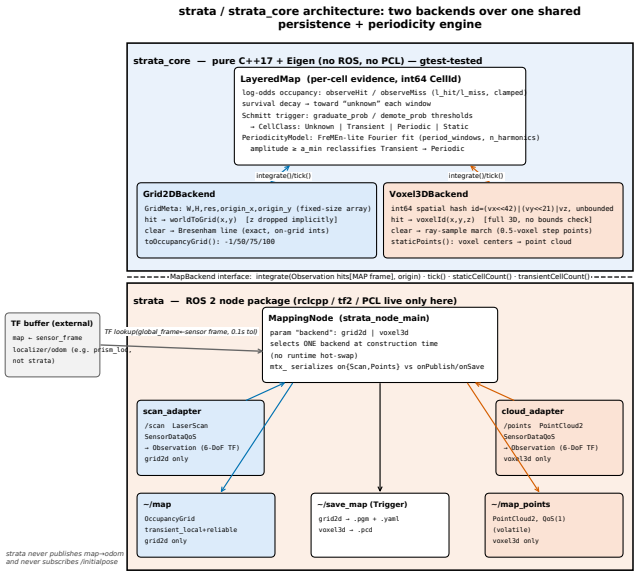

Fig. 1. Two-band architecture of STRATA. The upper band is strata_core (pure C++17 + Eigen, no ROS or PCL, gtest-tested): one shared LayeredMap persistence-and-periodicity engine, keyed by int64 CellId, feeds two sibling backend boxes Grid2DBackend and Voxel3DBackend. The lower band is the strata ROS 2 node, where MappingNode selects one backend at construction, its scan/cloud adapters convert messages to Observations in the map frame, and its timer publishes ~/map / ~/map_points and services ~/save_map. The two bands are split by the MapBackend interface line. The external TF arrow (map → sensor frame) enters the node from outside the diagram: STRATA consumes localization but never produces it.

```
struct Observation { std::vector<Eigen::Vector3d> hits; }; //
      endpoints, MAP frame
class MapBackend {
  virtual void integrate(const Observation& obs,
                         const Eigen::Vector3d&
                               sensor_origin_map) = 0;
  virtual bool tick() = 0;
  virtual std::size_t staticCellCount() const = 0;
  virtual std::size_t transientCellCount() const = 0;
};
```

Observation::hits is a flat list of 3D endpoints already expressed in the map frame; producing it from a sensor message and a sensor_to_map transform is the caller's job, so the backend never touches TF. integrate() performs one discrete map update from one sweep or cloud: for each hit it clears the free-space cells between sensor_origin_map and the hit via a backend-specific ray operation, then registers the hit itself as an occupied observation. tick() advances the temporal window and returns whether a window boundary was crossed. staticCellCount() and transientCellCount() are read-only introspection forwarded to the shared classifier.

Both concrete backends hold a LayeredMap member and satisfy the interface purely by translating world-frame geometry into int64 CellId keys that LayeredMap accumulates evidence for. This is the load-bearing design invari-

ant: the only per-backend code is point→id mapping and the free-space ray walk; the occupancy/persistence/periodicity classifier is shared by composition, not duplicated per dimension. LayeredMap and PeriodicityModel hold no coordinates, no resolution, and no origin — only integer ids — so the identical engine drives both 2D and 3D. The consequence, which the evaluation returns to, is that any cross-backend behavioral divergence or cost gap must originate in the geometry layer, because that is the only layer that differs.

B. Backend selection

Selection is a single string ROS parameter, backend ("grid2d" or "voxel3d"), read once in the node constructor. It is not polymorphic dispatch over a MapBackend*: the node holds both std::unique_ptr<Grid2DBackend> and std::unique_ptr<Voxel3DBackend> as separate members, and an if/else at construction instantiates exactly one of them, leaving the other null, and wires up the backend-specific parameters, publisher, and subscriber. We use a compile-time interface plus one runtime switch rather than dlopen-based plugin loading. At two backends, pluginlib [24] would add a registration and discovery mechanism whose cost is not repaid until the backend count grows; we defer that machinery until it is. Both branches read every LayeredMapParams field through a common helper, so the persistence and periodicity tuning surface is backend-independent by construction.

C. ROS 2 I/O contract

The node is strata::MappingNode, default node name strata. It looks up global_frame (default map) ← sensor frame on every incoming scan or cloud, at the message stamp, as a full 6-DoF Eigen::Isometry3d; there is no yaw-only flattening on the input side. Lookup failures are caught and rate-throttled rather than fatal, and that scan or cloud is dropped. STRATA is mapping-only: it neither subscribes to /initialpose nor broadcasts map→odom. Pose must already be published on TF by an external localizer. A single mutex serializes integration against publish and save.

The occupancy-value convention on ~/map and saved files is: unknown −1, transient 50, periodic 75, static 100. For voxel3d, only the static-cell set is emitted as points on ~/map_points.

TABLE I
ROS 2 input/output contract of the strata node.

Dir.	Name	Type / QoS	Condition
Sub	scan_topic (/scan)	LaserScan, SensorData	grid2d only
Sub	points_topic (/points)	PointCloud2, SensorData	voxel3d only
Pub	~/map	OccupancyGrid, QoS(1) TL reliable	grid2d only
Pub	~/map_points	PointCloud2, QoS(1) volatile	voxel3d only
Srv	~/save_map	Trigger, default	both (.pgm+.yaml / .pcd)
Timer	publish_period (1.0 s)	—	drives publishing, both

IV. The Layered Map Engine

This section specifies the engine as implemented in strata_core. All equations are transcribed from the source, not from the design specification; where the two disagree, §IV-H documents the difference. Symbols follow the notation used throughout the paper: ℓ is a cell's log-odds occupancy evidence, $p = \sigma(\ell)$ its occupancy probability, t the window (phase) index, H the harmonic count, T the base period, and λ the survival decay.

A. Per-frame accumulation and windowing

Each observeHit(id) / observeMiss(id) only increments the cell's window counters, window_hits or window_misses; it performs no log-odds arithmetic. Cells are created lazily on first touch, with $\ell = 0$, observations=0, and graduated=false. tick() advances the integration counter and closes a window on the interval boundary:

$$\text{closeWindow} \iff \big(\text{layer\_interval} \le 1\big) \vee \big(\text{integration\_count mod layer\_interval} = 0\big). \tag{1}$$

The layered update, endWindow(), then runs once over all live cells. With window counts h_w and m_w, each cell's window is scored:

$$\begin{aligned} \text{touched} &= [\,h_w > 0 \vee m_w > 0\,], \quad \text{occ} = [\,h_w > 0\,], \\ \text{free} &= [\,h_w = 0 \wedge m_w > 0\,]. \end{aligned} \tag{2}$$

A single hit outweighs any number of misses in the same window.

B. Log-odds persistence with survival decay

For touched cells only, the engine applies one log-odds increment (an inverse-sensor-model update in the sense of [14]), multiplies by the survival decay λ, and clamps:

$$\ell \leftarrow \text{clamp}\Big(\lambda\big(\ell + \underbrace{[\text{occ}]\, l_{\text{hit}} + [\text{free}]\, l_{\text{miss}}}_{\text{one increment}}\big),\ l_{\min},\ l_{\max}\Big), \tag{3}$$

$$\text{clamp}(x, a, b) = \min\big(b, \max(a, x)\big), \tag{4}$$

with occupancy probability $p = \sigma(\ell) = 1/(1 + e^{-\ell})$. The survival multiplier is a constant-rate special case of the Persistence Filter forgetting model [8].

Ordering note (load-bearing). The decay multiplies the already-incremented value, so the fresh hit or miss is itself attenuated by λ in the same window. This is not the classic decay-then-add order. One consequence is the decayed fixed point of repeated hits, $\ell^\star = \lambda\, l_{\text{hit}}/(1 -$

$\lambda) \approx 27.5$, which the clamp caps at $l_{\max} = 5$, giving $p_{\max} = \sigma(5) \approx 0.9933$. With the defaults, $p_{\text{grad}} = 0.8$ corresponds to $\ell \geq \ln 4 \approx 1.386$ and $p_{\text{prune}} = 0.05$ to $\ell < \ln(1/19) \approx -2.944$.

C. Schmitt-trigger graduation and demotion

After the log-odds update, p and the periodicity amplitude a (§IV-D) are recomputed for every cell, touched or not, and the graduated flag g is updated by a two-sided Schmitt trigger:

$$\text{graduate:} \quad \neg g \ \wedge \ \neg\text{periodic} \ \wedge \ p \geq p_{\text{grad}} \ \wedge \ \text{observations} \geq N_{\min} \ \Rightarrow \ g \leftarrow \text{true}, \tag{5}$$

$$\text{demote:} \quad g \ \wedge \ \big(p \leq p_{\text{dem}} \ \vee \ \text{periodic}\big) \ \Rightarrow \ g \leftarrow \text{false}. \tag{6}$$

The interval $[p_{\text{dem}}, p_{\text{grad}}]$ is the hysteresis band that suppresses flicker at the static boundary; the motivation is the same observation-consistency argument as in dynamic-object removal by reverting [9]. Two periodicity guards go beyond a bare threshold rule: !periodic blocks a strongly periodic cell from ever graduating to Static, and || periodic force-demotes a graduated cell that later reveals periodicity. The two rules cannot fight in one window, because graduation requires $\neg$periodic and $p \geq p_{\text{grad}} > p_{\text{dem}}$.

D. FreMEn-lite periodicity

In parallel with occupancy, each touched cell (when enable_periodicity) feeds an incremental Fourier model in the spirit of FreMEn [2]. With the per-window occupancy sample $v = [\text{occ}] \in \{0, 1\}$ and phase index t, gather accumulates:

$$n \mathrel{+}= 1, \quad S_0 \mathrel{+}= v, \tag{7}$$

$$\begin{aligned} C_k &\mathrel{+}= v \cos\big((k{+}1)\omega t\big), \\ S_k &\mathrel{+}= v \sin\big((k{+}1)\omega t\big), \quad k = 0..H{-}1, \end{aligned} \tag{8}$$

$$\omega = \frac{2\pi}{\max(1, T)}. \tag{9}$$

Here n counts touched windows fed to gather, not elapsed windows. The phase prediction at window t is

$$\hat{p}(t) = \text{clip}_{[0,1]}\left(\frac{S_0}{n} + \sum_{k=0}^{H-1}\Big[a_k \cos\big((k{+}1)\omega t\big) + b_k \sin\big((k{+}1)\omega t\big)\Big]\right), \tag{10}$$

with DC term equal to the empirical mean S_0/n and harmonic coefficients $a_k = 2C_k/n$, $b_k = 2S_k/n$. The dominant-harmonic amplitude and the periodicity test are

$$a = \begin{cases} 0, & n < T \\ \max\limits_{0 \leq k < H} \sqrt{a_k^2 + b_k^2}, & n \geq T \end{cases} \tag{11}$$

$$\text{periodic} = \text{enable\_periodicity} \ \wedge \ a \geq a_{\min}. \tag{12}$$

The $n \geq T$ gate means a sparsely observed cell needs T touches before its amplitude is trusted, which can span far more than T elapsed windows.

E. Cell-class state machine and pruning

At the end of each window, cells below confidence are erased:

$$\text{erase cell} \iff \neg g \ \wedge \ p < p_{\text{prune}} \ \wedge \ a < a_{\min}. \tag{13}$$

Static cells (g) and periodic cells ($a \geq a_{\min}$) are never pruned; pruning a cell also erases its FreMEn coefficients. A snapshot classifier reads out one of four states by a priority ladder:

$$\begin{gathered} \text{absent} \rightarrow \text{Unknown}; \quad g \rightarrow \text{Static}; \\ (\text{enable} \wedge a \geq a_{\min}) \rightarrow \text{Periodic}; \quad p \geq p_{\text{prune}} \rightarrow \text{Transient}; \\ \text{else} \rightarrow \text{Unknown}. \end{gathered} \tag{14}$$

The full transition table is Table II. Note that a present cell classifies Unknown when $p < p_{\text{prune}}$ and it is neither periodic nor graduated — the same condition under which it is generally pruned in the same window.

The complete per-window update is Algorithm 1.

Algorithm 1 endWindow(): one layered update per closed window, over all live cells

```
for each live cell c: # main update pass
    touched <- [h_w>0 or m_w>0]
    occ <- [h_w>0]
    free <- [h_w=0 and m_w>0]
    if touched:
        l <- l + [occ]*l_hit + [free]*l_miss # one log-odds increment
        l <- lambda * l # decay attenuates the fresh increment
        l <- clamp(l, l_min, l_max)
        observations <- observations + 1
        if enable_periodicity:
            gather(c, occ, t) # n+=1; S0+=v; Ck+=v cos((k+1)w t); Sk+=v sin((k+1)w t)
    p <- sigma(l); a <- amplitude(c) # recomputed for ALL cells, touched or not
    periodic <- enable_periodicity and a >= a_min
    if not g and not periodic and p >= p_grad and observations >= N_min:
        g <- true # graduate -> Static
    else if g and (p <= p_dem or periodic):
        g <- false # demote
    reset h_w <- 0, m_w <- 0
for each live cell c: # prune pass
    if not g and p < p_prune and a < a_min:
        erase c and its FreMEn coefficients
t <- t + 1
```

The window update is $O(N \cdot H)$ for N live cells and H constant, hence $O(N)$; it iterates all cells, including untouched ones, because the Schmitt and prune decisions read the recomputed p and a of every cell. Between window closes a frame is $O(1)$ per endpoint plus the backend ray cost.

TABLE II
Cell-class transitions, all evidence-driven and evaluated at window close.

From	To	Trigger
(implicit) Unknown	Transient	first observeHit/Miss: cell created, $\ell{=}0 \Rightarrow p{=}0.5 \geq p_{\text{prune}}$
Transient	Static	$p \geq p_{\text{grad}} \wedge \text{obs} \geq N_{\min} \wedge \neg$periodic (graduate)
Transient	Periodic	$a \geq a_{\min}$ (needs $n \geq T$ touched windows)
Transient	Unknown (erased)	$p < p_{\text{prune}} \wedge a < a_{\min} \wedge \neg g$ (prune)
Static	Transient / Periodic / Unknown	demote ($p \leq p_{\text{dem}} \vee$ periodic), then re-classified by ladder
Periodic	Transient / Unknown	a falls below $a_{\min}$ (then subject to prune)
Periodic	—	never graduates (!periodic guard), never pruned while $a \geq a_{\min}$
Static	—	never pruned while g

F. Geometry backends

The two backends differ only in how a world point becomes a CellId and how the free-space ray is walked; both delegate all evidence updates to the shared LayeredMap.

Grid2DBackend addresses a fixed-size, fixed-origin 2D array described by GridMeta = {width, height, resolution, origin_x, origin_y}. It buckets each hit's (x, y) (dropping z) into a row-major id $\text{gridCellId}(m, g_x, g_y) = g_y \cdot \text{width} + g_x$; points outside the array are dropped. Free space is cleared with integer Bresenham's line algorithm from the sensor cell to the hit cell, calling observeMiss on every intermediate cell strictly between the endpoints, so the hit cell receives only observeHit. Rendering to nav_msgs/OccupancyGrid initializes every cell to -1, then overwrites in ascending confidence order transient→50, periodic→75, static→100, so static wins any tie.

Voxel3DBackend has no fixed extent; the map grows through the hash map. Each axis is floor-divided by voxel_size and offset by $\text{kOff} = 1 \ll 20$ to keep the per-axis index non-negative, and the three 21-bit ($\text{kBits} = 21$) fields are packed into one 64-bit key, $\text{id} = (v_x \ll 42) \,|\, (v_y \ll 21) \,|\, v_z$; voxelCenter is the exact inverse, returning $(v + 0.5) \cdot$ voxel_size per axis. This gives O(1) hashing and a per-axis range far larger than any practical map. Free space is cleared by ray-sample marching, not geometric voxel traversal: the ray is subdivided into $\lceil \text{len}/(0.5 \cdot \text{voxel\_size}) \rceil$ half-voxel steps, and each interior sample is hashed to its containing voxel and marked observeMiss. This is simpler than exact traversal but can over-sample a long ray and can skip a thin voxel when the step count rounds down. The hit is always observeHit, with no bounds check because the hash grows to fit. staticPoints() maps the static-cell set through voxelCenter to a point cloud. Table III contrasts the two.

G. Parameters

Table IV lists the engine's tunable parameters with their defaults, which are identical between the struct definition and both shipped YAML files. Geometry and node parameters (grid_width 400, grid_height 400, grid_resolution 0.05 m, grid_origin_{x,y} −10.0 m for grid2d; voxel_size 0.2 m for voxel3d; frame names, topics, publish_period 1.0 s) are not part of the engine math.

Per-cell state is compact: CellEvidence is 24 bytes, plus roughly 32 bytes of unordered_map node overhead, giving the ∼56 B/cell footprint reported in the evaluation when periodicity storage is excluded. A FreMEn-tracked cell adds a Coeff record (∼96 B payload at $H = 2$), allocated only for cells touched at least once with periodicity enabled.

H. Implementation versus specification

We document the code that ships, not the design prose. Three points where the implementation diverges from SPEC.md are surfaced here rather than smoothed over, because each changes the operational semantics.

SPEC-DIFF #1 (forgetting is coupled to re-observation). The specification states that decay happens "every window" and that "a cell that stops being observed decays out." In code, the entire log-odds-and-clamp block of §IV-B is inside if (touched). An untouched cell — out of the sensor field of view, with neither a hit nor a miss that window — does not decay; its ℓ freezes. Forgetting therefore requires being re-observed as free (a miss is a touch), so clutter fades only while it stays in the field of view, and anything that leaves the field of view persists indefinitely. This is the most consequential difference and is restated as an operational limitation in the discussion.

SPEC-DIFF #2 (periodicity guards absent from the formal block). The specification's §3.4 pseudocode omits the periodicity guards entirely, listing graduate as !graduated && p >= graduate_prob && observations >= min_observations and demote as graduated && p <= demote_prob. The !periodic and || periodic terms actually in the code (§IV-C) appear only in prose and comments. A method transcribed verbatim from that block would be wrong; the equations above follow the code.

SPEC-DIFF #3 (amplitude gate counts touched windows, not elapsed time). The specification says amplitude is zero before period_windows windows have elapsed. The code gates on $n < T$, where n is the count of touched windows (calls to gather), not elapsed wall or window time. A sparsely observed cell needs T touches, which can span far more than T windows — the mechanism behind the low-duty periodicity miss reported in the evaluation.

TABLE III
The two geometry backends. All persistence and periodicity logic is shared; only these two rows of behavior differ.

	Grid2DBackend	Voxel3DBackend
Cell addressing	2D array index via GridMeta (fixed $W \times H$, fixed origin)	3D spatial hash, unbounded, int64 packed key
Hit dimensionality	x, y (z dropped by projection)	x, y, z (full volumetric)
Free-space ray	Bresenham line (exact, on-grid)	ray-sample march at 0.5-voxel step (approximate, off-grid)
Growth	none — pre-sized at construction	unbounded — hash grows with exploration
Output	nav_msgs/OccupancyGrid (dense −1/50/75/100)	static voxel centers → PointCloud2

TABLE IV
Engine parameters and defaults (struct and params/*.yaml agree).

Name	Default	Meaning
layer_interval	10	integration ticks per layered-update window
l_hit	0.85	log-odds increment on an occupied window
l_miss	−0.4	log-odds increment on a free window
l_min	−5.0	clamp lower bound (log-odds)
l_max	5.0	clamp upper bound (log-odds)
survival_decay (λ)	0.97	Persistence-Filter forgetting multiplier, per window
graduate_prob (p_{grad})	0.8	p to promote → Static
demote_prob (p_{dem})	0.45	p at/below which Static demotes (hysteresis floor)
min_observations ($N_{\min}$)	3	min touched windows before a cell may graduate
prune_prob (p_{prune})	0.05	erase non-static, non-periodic cell below this p
enable_periodicity	true	run the FreMEn model
periodic_amplitude_min ($a_{\min}$)	0.3	dominant-harmonic magnitude to classify Periodic
period_windows (T)	24	FreMEn base period; also amplitude-validity gate ($n \geq T$)
n_harmonics (H)	2	Fourier harmonics tracked per cell

V. Evaluation (synthetic)

A. Setup

All four experiments (E1–E4) are driven by a standalone harness in paper/experiments/ that links directly against the strata_core sources — no ROS 2, no ament, no colcon. Reproduction from the repository root is a single call, bash paper/experiments/run_all.sh, which configures and builds strata_core and the four harness executables, runs each one, and writes the CSVs in paper/experiments/results/ from which every number and figure in this section is taken; strata_core's own ctest suite (1/1) is checked first, so the harness is only trusted once the unit-level behavior it builds on is confirmed. Every stochastic experiment (E1–E3) seeds a single std::mt19937 from the fixed constant kSeed = 12345, recorded in each CSV's #seed row, so the reported precision/recall/F1/flicker/TPR/FPR numbers are exactly reproducible. E4 additionally reads the wall clock to time integrate()/endWindow() calls; its absolute microsecond figures are therefore machine-dependent, while the live-cell counts it also reports are deterministic (they follow only from ray geometry, not timing).

The four experiments map onto the two thesis claims of §III–IV (C2, C5) differently. E1 and E4 probe backend-behavior equivalence: both feed identical hit/miss streams through Grid2DBackend and Voxel3DBackend and compare the outcome, so any divergence is attributable only to the geometry layer (point→CellId mapping and the free-space ray walk), since the classifier code both backends call is the same LayeredMap instance type. E2 and E3, by contrast, drive LayeredMap/PeriodicityModel directly, with no backend in the loop at all — a choice that is itself an argument for C1: because persistence, hysteresis, and periodicity are implemented once, independent of CellId provenance, characterizing them against LayeredMap directly is valid for whichever backend supplies the cell ids in production. Each harness constructs its own LayeredMapParams in code rather than loading the shipped YAML, so several values deviate from the Table IV production defaults; Table V lists every deviation, per experiment, so the runs are reproducible without reading the harness source. Three deviations are common to interpreting the results below: all four harnesses run at layer_interval 1 (one window per integration tick, against the production default of 10), so "window" and "frame" coincide in every reported count; E1, E3, and E4 disable periodicity to isolate the persistence layer (E2 is the only run that exercises the FreMEn model); and E4 and E2 both set survival_decay 1.0 (no forgetting). E2 in particular raises all six of the log-odds/graduation defaults it touches (l_{hit} 1.0, l_{miss} −1.0, λ 1.0, p_{grad} 0.9, p_{dem} 0.4, $N_{\min}$ 5) to drive its detected-versus-pruned classification, and E3 lowers prune_prob to 0.01 to keep noisy walls alive; these are the values actually compiled into each harness.

B. E1 — Static-layer map quality vs. time

Setup. A ground-truth static cross of 161 cells (segment y=60, $x \in [20, 100]$ and x=60, $y \in [20, 100]$) is hit every window, alongside transient movers at three densities — low/med/high = 5/25/100 fresh random cells per window — over 40 windows, run against both a 120×120 (resolution 1.0) Grid2DBackend and a 1.0-voxel Voxel3DBackend, with periodicity disabled to isolate the persistence layer. Static-set precision/recall/F1 against the 161-cell wall set is logged per window (results/e1_static_quality.csv).

TABLE V
Per-experiment deviations from the Table IV production defaults. "—" = default unchanged; "swept" = varied across the E3 configuration grid ($\lambda \in \{0.90, 0.97, 1.0\}$, graduate_prob $\in \{0.6, 0.7, 0.8, 0.9\}$, demote_prob $\in \{0.3, 0.5, \text{graduate\_prob}\}$); "n/a" = inert because periodicity is disabled in that run.

Parameter (default)	E1	E2	E3	E4
layer_interval (10)	1	1	1	1
enable_periodicity (true)	false	true	false	false
l_{hit} (0.85)	—	1.0	—	—
l_{miss} (−0.4)	—	−1.0	—	—
survival_decay (0.97)	—	1.0	swept	1.0
graduate_prob (0.8)	—	0.9	swept	—
demote_prob (0.45)	—	0.4	swept	—
min_observations (3)	—	5	—	—
prune_prob (0.05)	—	—	0.01	—
period_windows (24)	n/a	8	n/a	n/a
n_harmonics (2)	n/a	3	n/a	n/a

TABLE VI
E1 — final static-set precision/recall/F1, both backends, all three clutter densities.

backend	density	recall = 1	prec. @39	F1 @39
grid2d	low (5/w)	window 3	1.0000	1.0000
grid2d	med (25/w)	window 3	0.9938	0.9969
grid2d	high (100/w)	window 3	0.9253	0.9612
voxel3d	low (5/w)	window 3	1.0000	1.0000
voxel3d	med (25/w)	window 3	0.9938	0.9969
voxel3d	high (100/w)	window 3	0.9758	0.9877

With $N_{\text{min}} = 3$, a cell needs three touched windows before it is eligible to graduate; the CSV confirms recall jumps from 0 to 1.0 exactly at the third window (window index t=2) for every one of the six backend×density combinations, and holds at 1.0 through w=39 in every case — no wall cell is ever demoted by clutter. The only errors are a handful of false-positive statics that appear when a random mover happens to re-hit the same cell often enough to graduate on its own; at low density (5 movers/window) this never happens at all, and precision stays exactly 1.0 through w=39 for both backends. At medium density it saturates at a single spurious cell (precision floors at 0.9938 and never falls further), but at high density (100 movers/window) false statics keep accumulating window over window — from 162 predicted cells at w=17 (grid2d) / w=25 (voxel3d) to 174 (grid2d) / 165 (voxel3d) by w=39 — giving the only material gap between backends: precision 0.9253 (grid2d) vs. 0.9758 (voxel3d) at the final window. Both backends run the identical LayeredMap graduation rule on the identical mover stream; the gap traces to geometry, not the classifier — Grid2DBackend projects every hit onto a shared 2D (x, y) cell (dropping z), so independent movers collide into the same cell more often than in Voxel3DBackend's full 3D hash, and each collision is exactly the accumulation event that produces a spurious static.

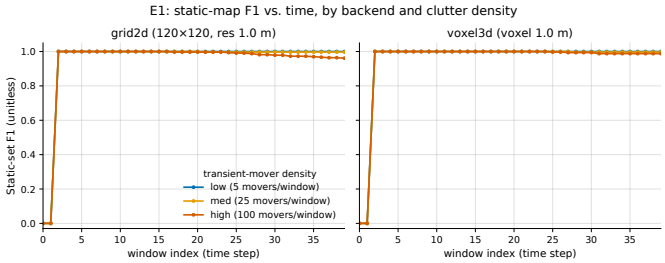

Fig. 2. E1: static-set F1 vs. window index, one panel per backend, one line per clutter density. Both backends recover F1≈1.0 within three windows; the 100-movers/window regime is the only one with a visible precision/F1 dip, and the dip is larger for grid2d.

TABLE VII
E2 — per-cell classification against ground truth ($a_{\text{min}} = 0.3$); Periodic TPR = 2/3, FPR = 1/5.

cell	ground truth	final class	ampl.	note
door_p8_4on4off (50% duty)	periodic	Periodic	0.653	correct
door_p4_2on2off (50% duty)	periodic	Periodic	0.707	correct
door_p8_2on6off (25% duty)	periodic	Transient	—	miss
wall_constant	non-periodic	Static	≈0	correct
aperiodic_1	non-periodic	Periodic	0.329	FP
aperiodic_0,_2,_3	non-periodic	Transient	0.14–0.25	correct

C. E2 — Periodicity detection

Setup. Three periodic doors (p8 4-on/4-off, p8 2-on/6-off, p4 2-on/2-off, all detectable against the FreMEn base period T=8 with H=3 harmonics), a constant wall, and four deterministic Bernoulli(0.5) aperiodic movers as negative controls, are driven through the real LayeredMap pipeline for 64 windows. Reported: per-cell final class against ground truth (Periodic TPR/FPR), and FreMEn dominant-harmonic amplitude vs. observation length (results/e2_classification.csv, results/e2_amplitude_vs_length.csv, results/e2_summary.csv).

Both 50%-duty doors are detected cleanly: amplitude 0.653 and 0.707 clear $a_{\text{min}} = 0.3$ by a wide margin. The one miss, the 25%-duty door (door_p8_2on6off), is a prune/maturity race, not a modeling failure: the notes accompanying this harness report that the real pipeline prunes this cell in roughly 10 of its 64 windows — each 6-window vacant stretch lets its log-odds fall below p_{prune} while its FreMEn amplitude is still 0 (the $n \geq T$ gate has not yet been reached), erasing its accumulator state before periodicity can mature. The CSV's ref_amplitude column — a non-pruning reference PeriodicityModel fed the identical occupancy stream — reads 0.46194 for this same cell, comfortably above a_{min},

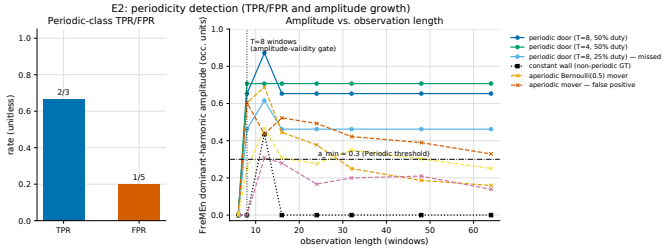

Fig. 3. E2: (left) Periodic-class TPR/FPR bars; (right) FreMEn dominant-harmonic amplitude vs. observation length for all 8 probe cells, with the $a_{\min} = 0.3$ threshold and the $n \geq T$ validity gate marked — the two 50%-duty doors clear the threshold cleanly, the 25%-duty door is pruned before it matures, and one aperiodic mover produces the false positive.

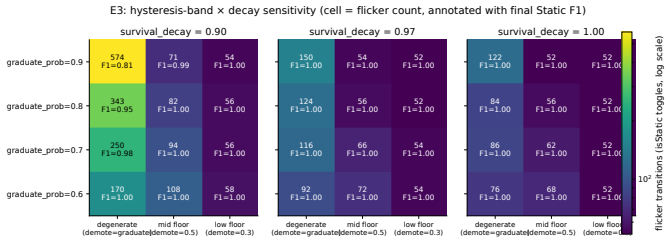

Fig. 4. E3: three heatmap panels (one per survival_decay); rows = graduate_prob, columns = hysteresis-band width, cells colored by flicker-transition count (log scale) and annotated with the raw count and final F1 — the degenerate (zero-band) column drives both the worst flicker and the worst F1 at every decay setting.

confirming the signal itself is periodic and detectable; the miss is specifically an interaction between pruning and the touched-window amplitude gate. The lone false positive, aperiodic_1 at amplitude 0.329, is a Bernoulli(0.5) cell whose Fourier energy over 64 windows happens to clear the 0.30 threshold by a small margin (0.029 above $a_{\min}$) — the kind of chance alignment expected from a fixed-length random negative control rather than a systematic classifier flaw. The amplitude-vs-length data also confirms the validity gate operates as specified: for door_p8_4on4off, amplitude is exactly 0 at observation length 6 ($n < T$=8) and jumps to 0.653 at length 8 ($n \geq T$), where it then stabilizes.

D. E3 — Sensitivity: hysteresis band and decay

Setup. 50 ground-truth wall cells are observed occupied with probability 0.6 each window (noisy) alongside 10 fresh movers/window, over 80 windows, with the identical noise sequence replayed for every configuration in the sweep. The sweep spans graduate_prob $\in \{0.6, 0.7, 0.8, 0.9\}$ × demote_prob (a low floor 0.3, a mid floor 0.5, and the degenerate demote_prob == graduate_prob, i.e. zero hysteresis band) × survival_decay $\in \{0.90, 0.97, 1.0\}$ — 36 configurations total. Reported: final Static-layer F1 and the count of per-window isStatic flicker transitions (results/e3_sensitivity.csv).

TABLE VIII
E3 — selected sweep configurations (full 36-row sweep in the CSV); identical replayed noise throughout.

grad.	dem.	band	decay	flicker	F1
0.9	0.9 (deg.)	0	0.90	574	0.810
0.8	0.8 (deg.)	0	0.90	343	0.947
0.7	0.7 (deg.)	0	0.90	250	0.980
0.9	0.9 (deg.)	0	0.97	150	1.000
0.9	0.9 (deg.)	0	1.00	122	1.000
0.9	0.3	0.6	0.90	54	1.000
0.9	0.3	0.6	0.97	52	1.000
0.9	0.3	0.6	1.00	52	1.000

Removing hysteresis is the single largest effect in the sweep: at graduate_prob = demote_prob = 0.9, survival_decay = 0.90 (the worst degenerate configuration), the wall layer flickers 574 times and loses 19% of its final F1 (0.810, recall collapses to 0.68), while a same-noise, same-decay run with the hysteresis band widened to 0.6 (demote_prob = 0.3) flickers only 54 times at F1 1.000 — the band alone accounts for a $> 10\times$ reduction in flicker. The degenerate (zero-band) configurations are also internally monotonic in threshold: at survival_decay = 0.90, flicker rises with the coincident threshold value (250 at 0.7, 343 at 0.8, 574 at 0.9) even though a higher threshold is ordinarily the more conservative, more "static" setting — a threshold pair with no gap between graduate and demote is not a substitute for a hysteresis band, regardless of where that pair sits. survival_decay is the second-order effect: holding the worst degenerate threshold pair (0.9/0.9) fixed, flicker falls monotonically as forgetting slows — 574 at λ=0.90, 150 at λ=0.97, 122 at λ=1.0 — because faster decay lets noisy log-odds cross the (here, coincident) threshold more often. With a wide hysteresis band, survival_decay barely matters (52–54 flicker across all three decay settings at the best band): hysteresis, not decay, is what keeps the static layer stable.

E. E4 — Throughput and memory

Setup. 150 random hits per window over 20 windows, for Grid2DBackend vs. Voxel3DBackend at three nominal extents (100/250/500, resolution/voxel size 1.0), periodicity disabled. Reported: mean per-integrate() call latency, mean per-endWindow() cell-update rate, final live-cell count, and estimated memory at the fixed 56 B/cell footprint documented in §IV (CellEvidence 24 B + ≈32 B unordered_map node overhead, with FreMEn storage excluded since periodicity is off in this run) (results/e4_throughput.csv).

At every tested extent, Grid2DBackend's integrate() call is cheaper than Voxel3DBackend's at the identical

TABLE IX
E4 — per-call latency, throughput, live-cell count, and estimated memory, both backends, all three extents. Absolute μs are single-machine wall-clock samples; live-cell counts (and the ratios derived from them) are geometry-determined and reproducible.

backend	extent	μs/intg.	updates/s	live cells	memory
grid2d	100^2	320.8	9.41×10^6	8,553	479 KB
voxel3d	100^2	4,796.8	7.06×10^6	85,399	4.78 MB
grid2d	250^2	1,702.6	1.07×10^7	45,657	2.56 MB
voxel3d	250^2	18,469.5	4.46×10^6	295,300	16.5 MB
grid2d	500^2	6,548.9	8.64×10^6	150,096	8.41 MB
voxel3d	500^2	35,958.3	4.45×10^6	661,410	37.0 MB

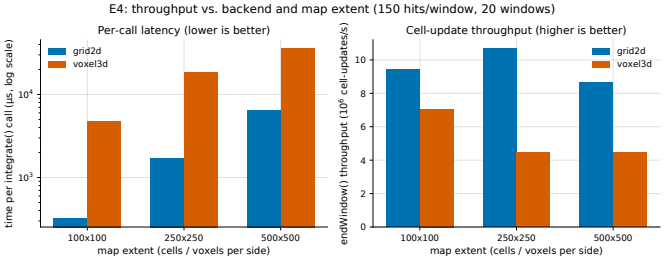

Fig. 5. E4: grouped bars by backend × extent — per-integrate() latency in μs (log scale, left) and endWindow() cell-update throughput in 10^6 updates/s (right). grid2d is cheaper per call than voxel3d at every tested extent; the gap tracks voxel3d's larger live-cell count.

nominal extent and hit rate — by 4,796.8/320.8 ≈ 14.9× at 100^2, 18,469.5/1,702.6 ≈ 10.8× at 250^2, and 35,958.3/6,548.9 ≈ 5.5× at 500^2, the multiple shrinking as extent grows. The live-cell counts explain why: Voxel3DBackend ends each run holding 85,399/8,553 ≈ 10.0×, 295,300/45,657 ≈ 6.5×, and 661,410/150,096 ≈ 4.4× as many cells as Grid2DBackend at the same extent, because its free-space ray-sample march (§IV-F) walks a full 3D volume and spawns far more traversed voxels than Grid2DBackend's exact 2D Bresenham clear over the same nominal footprint — the same geometric asymmetry that drives the latency gap also drives the cell-count gap, and both narrow together as the extent grows and the ray paths (and hence the per-ray voxel counts) become a smaller fraction of the total volume. Memory is exactly what a flat 56 B/cell predicts: estimated memory tracks live-cell count linearly across every row of Table IX with no additional constant, topping out at 37.0 MB for the largest 3D configuration tested (661,410 cells × 56 B) — inexpensive even at the largest map in this sweep. Absolute microsecond values here are single-run wall-clock measurements on one machine and are not claimed to be portable; the live-cell counts, and the ratios and memory figures derived from them, follow only from ray/hash geometry and reproduce under the fixed seed regardless of machine.

F. Synthesis

Across E1–E4, the two backends behave as one engine wearing two geometries, exactly as C1 asserts by construction: E1's near-identical F1 trajectories (both backends graduate the wall by the third window and hold recall 1.0 through w=39) and E4's flat 56 B/cell tracking live-cell count are the empirical face of "no per-dimension logic or cost," since LayeredMap and PeriodicityModel — exercised directly and identically in E2 and E3 — never see which backend produced the CellId they are updating. The two places where the backends do diverge are both traced, not merely asserted, to geometry rather than to the shared classifier: E1's high-clutter precision gap (0.9253 vs. 0.9758) follows from Grid2DBackend's 2D z-projection colliding independent movers into shared cells more often than Voxel3DBackend's full-3D hash, and E4's per-call latency gap (5.5–14.9×, narrowing with extent) follows from Voxel3DBackend's free-space ray-sample march spawning 4.4–10.0× more live voxels than Grid2DBackend's exact line clear at the same nominal extent. E2 and E3, run once against LayeredMap with no backend involved at all, characterize the shared engine's own behavior — clean 50%-duty periodicity detection with a diagnosed prune/maturity-race failure mode at low duty cycle, and hysteresis as the dominant flicker suppressor, decay a secondary one — and that characterization is, by the same construction argument, valid for whichever backend feeds it cell ids in production.

VI. Discussion and Limitations

A. What has and has not been shown

The evaluation in §V characterizes a shipped engine; it does not validate a deployed system. Every number in this paper — E1's precision/recall curves, E2's TPR/FPR, E3's flicker counts, E4's throughput and memory — comes from a deterministic, seeded synthetic harness compiled directly against strata_core, driving hand-authored hit/miss patterns through the real LayeredMap/PeriodicityModel code. No real sensor, no real robot motion, and no field deployment enters any of these results. This is a statement of present scope, not a hidden gap: the closest published systems in intent (ELite [4], LTmapper [5]) and the closest production baseline (Berrio et al. [7]) all report multi-day or multi-session field evidence that STRATA does not yet have. STRATA is, so far, validated the way a library is validated — by unit and characterization tests against known ground truth — not the way a robot behavior is validated, by miles driven.

A second, narrower scope limit sits inside the "lifelong" framing itself. STRATA implements single-session temporal persistence: a cell's class is a running function of its own log-odds, observation count, and Fourier accumulators, updated window by window within one continuous mapping run. There is no session boundary, no re-localization into a prior map, and no cross-session

registration — a graduated Static cell simply is the current map, with its history implicit in accumulator state rather than explicitly stored. This is a strictly smaller claim than LT-mapper's live/meta/delta session management or Yang et al.'s versioned base-map-plus-deltas [13], both of which let a robot query or reconstruct a past session. STRATA cannot do either; it only ever has "now."

Finally, the paper does not claim bit-identical cross-backend behavior. §V already frames grid2d and voxel3d as equivalent to within geometry-induced differences rather than identical, and E1's clutter-driven precision gap (0.9253 vs. 0.9758 at 100 movers/window) and E4's 5.5–14.9× per-integrate() cost gap are exactly those differences, measured rather than glossed over.

B. The out-of-FOV forgetting caveat is an operational limitation, not a footnote

§IV-H documents SPEC-DIFF #1 as an implementation fact: decay is applied only inside the touched branch of endWindow(), so a cell that leaves the sensor's field of view stops decaying entirely — its log-odds value freezes at whatever it last reached, rather than relaxing toward the unknown prior. Restated as an operational consequence: a transient object that is observed long enough to accumulate strong occupied evidence, and then permanently exits the sensor's coverage (a box dragged out of a corridor the robot no longer revisits, a door propped open just outside the LiDAR's usual sweep), retains its evidence indefinitely. STRATA's forgetting is coupled to re-observation, not to elapsed time. In a bounded, frequently re-swept environment — the setting E1–E4 implicitly assume, and the setting the canonical Integration.WallStaticMoverTransientDoorPeriodic scenario exercises — this rarely matters, because clutter that matters is clutter the robot keeps driving past. In a sparsely-revisited environment it is a real failure mode the current implementation does not address, and it is one of the concrete items in the future-work list below rather than an incidental detail.

C. Design trade-offs owned honestly

STRATA makes three simplifications relative to richer prior formalisms, each adopted deliberately and each with a known cost.

A global scalar decay rate, not a per-cell-learned one. survival_decay is a single tunable constant shared by every cell, applied as in §IV-B. The Persistence Filter's survival-time posterior [8] and the Markov-chain occupancy models of Tipaldi et al. and Saarinen et al. [15], [16] instead let each cell (or region) learn its own forgetting rate from observed transition statistics — a wall in a rarely-disturbed alcove and a doormat in a busy doorway would decay at different, empirically fit rates. STRATA's one-parameter simplification is what E3 characterizes: a single survival_decay value, uniformly applied, already buys most of the achievable stability once paired with hysteresis (F1 1.000 for the wide-band configuration at both decay=0.90 and decay=1.0), so the paper does not claim the uniform rate is a limitation in the tested regime — only that it is a strictly less expressive model than the per-cell-learned alternative, and a scene with sharply heterogeneous dynamics per cell is untested.

A flat voxel hash, not an octree. Voxel3DBackend stores live voxels in an unordered_map<CellId, CellEvidence> keyed by a packed int64, with no multi-resolution compression. OctoMap [18] and UFOMap [20] instead exploit spatial coherence — large free or occupied regions collapse into single octree nodes — trading a more complex tree structure for asymptotically better memory scaling on sparse or large-extent maps. E4 measures the cost of this choice directly: voxel3d holds 4.4–10.0× the live-cell count of grid2d at matched scene extent because every sampled free-space point along a ray becomes its own hash entry, and per-cell footprint is a flat 56 B regardless of spatial redundancy. The trade is deliberate — O(1) insert/lookup with no tree-rebalancing logic keeps the backend simple enough to unit-test the way §III describes — but it means STRATA's voxel3d backend will not scale as gracefully as an octree-backed map to very large, sparsely-occupied 3D extents.

Two backends behind an if/else, not a plugin registry. §III argues, and the Related Work rejection of pluginlib [24] restates, that a compile-time MapBackend interface with two concrete unique_ptr members selected by one runtime string parameter is the right amount of mechanism for exactly two backends. This is a design bet, not a measured result: it costs nothing today, but it does not generalize — a third backend (e.g. a signed-distance or hybrid representation) would need either a third branch bolted onto the same if/else (tolerable once, ugly repeated) or a migration to pluginlib-style dynamic loading. The paper takes no position on which is correct beyond two backends; it only argues the current structure is not premature generalization at the current backend count.

Honesty has a cost the paper pays openly. Choosing the systems/tool framing over an accuracy-SOTA framing means every claim in §V is scoped to "what the shipped code measurably does," not "how well STRATA localizes or maps relative to a competing system." No baseline comparison against OctoMap, ELite, or LT-mapper is run in this paper — none is designed as an apples-to-apples ROS-free unit-testable core, so no fair single-number comparison exists yet. The E1–E4 harness is offered instead as a replayable protocol (§VII) a future study could run against an alternative implementation, rather than a leaderboard result against one run today.

D. When not to use STRATA

STRATA's boundaries, stated plainly: it is not a substitute for SLAM — it performs no pose estimation, scan matching, or loop closure, and it consumes rather than

produces the map → sensor transform, so a deployment without an external localizer (prism_loc or otherwise already publishing that TF) has no path to a usable map. It is not a substitute for multi-session mapping — a robot that needs to compare Monday's map against Friday's, or replay a specific past session, needs LT-mapper- or Yang et al.-style session versioning [5], [13] that STRATA does not provide. And it is not a substitute for semantic or object-level reasoning — the cell-class state machine of §IV-E has no notion of "this cluster of cells is one dynamic agent," unlike Khronos's scene graph [11] or ERASOR's per-object removal [17]. Within those boundaries — a single continuous mapping session behind an external localizer, at the raw occupancy-cell level, in 2D or 3D — STRATA is the intended fit.

E. Future work

Four items follow directly from the limitations above, in the order they would be tackled: (1) real-robot and multi-session field validation, closing the synthetic-only gap of §VI.A, on both a 2D wheeled platform and a 3D-LiDAR platform to exercise both backends under real sensor noise; (2) a per-cell-learned decay rate along the lines of iMac/Tipaldi [15], [16], replacing the global survival_decay scalar characterized above with region- or cell-adaptive forgetting; (3) resolving the prune/maturity race identified in E2 — the 25%-duty periodic door is pruned during its own vacant stretches before its FreMEn amplitude has enough touched-window support to mature (§V) — by either exempting recently-active cells from pruning or feeding a running amplitude estimate into the prune decision itself; and (4) decoupling forgetting from re-observation, i.e. fixing SPEC-DIFF #1 so an untouched cell's log-odds relaxes toward the unknown prior on a wall-clock or tick basis rather than only ever decaying when re-touched, closing the out-of-FOV persistence gap without changing the touched-cell update path E1–E4 already characterize.

VII. Conclusion

STRATA packages lifelong occupancy mapping's persistence-and-periodicity logic — log-odds accumulation, Persistence-Filter-style survival decay, Removert-motivated Schmitt-trigger hysteresis, ReFusion-style negative evidence from free-space ray clearing, and FreMEn-lite periodicity detection — into one geometry-free LayeredMap/PeriodicityModel engine that drives two pluggable geometries, a 2D occupancy grid and a 3D voxel hash, behind a single MapBackend interface and a one-parameter runtime switch. The only code that differs per backend is point-to-cell-id mapping and the free-space ray walk (§III–§IV); the classifier that decides Static, Periodic, Transient, or Unknown is written once and shared by composition. That engine builds and unit-tests with a plain C++17 + Eigen toolchain, independent of ROS 2, DDS, or PCL (§III), and its behavior is characterized — not merely asserted — by a seeded, reproducible synthetic harness: near- identical static-layer quality across backends up to a clutter-induced, geometrically-explained precision gap (E1), clean detection of 50%-duty periodicity with a diagnosed failure mode at low duty cycles (E2), hysteresis as the dominant stabilizer against flicker (E3), and a flat per-cell memory footprint with a 5.5–14.9× backend cost gap attributable entirely to 3D free-space voxel proliferation, not to the shared engine (E4). None of this replaces field validation, multi-session mapping, or semantic reasoning — §VI states those boundaries explicitly — but within them, STRATA is offered as a small, dependency-light module other systems can sit on top of rather than a competing full lifelong-SLAM pipeline.

A. Reproducibility

STRATA is open source under the Apache-2.0 license at github.com/kjungmo/strata, targeting ROS 2 Humble. The strata_core package alone builds and unit-tests without ROS installed (cmake -S strata_core -B build -DSTRATA_CORE_BUILD_TESTS=ON && ctest, §III). All figures and tables in §V regenerate from the CSVs the harness itself writes: bash paper/experiments/run_all.sh configures and builds the four E1–E4 executables against strata_core, runs them with the fixed harness seed kSeed = 12345, and writes paper/experiments/results/*.csv; a follow-up paper/experiments/plot/run_all_plots.sh regenerates every figure from those CSVs. strata_core's own ctest suite (1/1) is checked before the harness output is trusted. We invite integration reports, alternative backend implementations replayed against the Integration.WallStaticMoverTransientDoorPeriodic reference scenario, and pairing with an external localizer such as prism_loc for end-to-end deployment.

References

[1] P. Biber and T. Duckett, "Dynamic maps for long-term operation of mobile service robots," in Robotics: Science and Systems I (RSS), Cambridge, MA, USA, 2005.

[2] T. Krajník, J. P. Fentanes, J. M. Santos, and T. Duckett, "FreMEn: Frequency map enhancement for long-term mobile robot autonomy in changing environments," IEEE Transactions on Robotics, vol. 33, no. 4, pp. 964–977, 2017.

[3] T. Krajník, J. P. Fentanes, G. Cielniak, C. Dondrup, and T. Duckett, "Spectral analysis for long-term robotic mapping," in 2014 IEEE International Conference on Robotics and Automation (ICRA), Hong Kong, China, 2014, pp. 3706–3711.

[4] H. Gil, D. Lee, G. Kim, and A. Kim, "ELite: Ephemerality meets LiDAR-based lifelong mapping," in 2025 IEEE International Conference on Robotics and Automation (ICRA), 2025.

[5] G. Kim and A. Kim, "LT-mapper: A modular framework for LiDAR-based lifelong mapping," in 2022 IEEE International Conference on Robotics and Automation (ICRA), 2022.

[6] M. Labbé and F. Michaud, "RTAB-map as an open-source lidar and visual SLAM library for large-scale and long-term online operation," Journal of Field Robotics, vol. 36, no. 2, pp. 416–446, 2019.

[7] J. S. Berrio, S. Worrall, M. Shan, and E. Nebot, "Long-term map maintenance pipeline for autonomous vehicles," IEEE Transactions on Intelligent Transportation Systems, vol. 23, no. 8, pp. 10 427–10 440, 2021.
[8] D. M. Rosen, J. Mason, and J. J. Leonard, "Towards lifelong feature-based mapping in semi-static environments," in 2016 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2016, pp. 1063–1070.
[9] G. Kim and A. Kim, "Remove, then revert: Static point cloud map construction using multiresolution range images," in 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020, pp. 10 758–10 765.
[10] E. Palazzolo, J. Behley, P. Lottes, P. Giguère, and C. Stachniss, "ReFusion: 3d reconstruction in dynamic environments for RGB-D cameras exploiting residuals," in 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2019, pp. 7855–7862.
[11] L. Schmid, M. Abate, Y. Chang, and L. Carlone, "Khronos: A unified approach for spatio-temporal metric-semantic SLAM in dynamic environments," in Robotics: Science and Systems (RSS), 2024.
[12] D. Meyer-Delius, J. Hess, G. Grisetti, and W. Burgard, "Temporary maps for robust localization in semi-static environments," in 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Taipei, Taiwan, 2010, pp. 5750–5755.
[13] L. Yang, S. M. Prakhya, S. Zhu, and Z. Liu, "Lifelong 3d mapping framework for hand-held & robot-mounted LiDAR mapping systems," IEEE Robotics and Automation Letters, vol. 9, no. 11, pp. 9446–9453, 2024.
[14] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA: MIT Press, 2005.
[15] G. D. Tipaldi, D. Meyer-Delius, and W. Burgard, "Lifelong localization in changing environments," The International Journal of Robotics Research, vol. 32, no. 14, pp. 1662–1678, 2013.
[16] J. Saarinen, H. Andreasson, and A. J. Lilienthal, "Independent Markov Chain occupancy grid maps for representation of dynamic environments," in 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2012, pp. 3489–3495.
[17] H. Lim, S. Hwang, and H. Myung, "ERASOR: Egocentric ratio of pseudo occupancy-based dynamic object removal for static 3d point cloud map building," IEEE Robotics and Automation Letters, vol. 6, no. 2, pp. 2272–2279, 2021.
[18] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3d mapping framework based on octrees," Autonomous Robots, vol. 34, no. 3, pp. 189–206, 2013.
[19] M. Nießner, M. Zollhöfer, S. Izadi, and M. Stamminger, "Real-time 3d reconstruction at scale using voxel hashing," ACM Transactions on Graphics, vol. 32, no. 6, pp. 169:1–169:11, 2013.
[20] D. Duberg and P. Jensfelt, "UFOMap: An efficient probabilistic 3d mapping framework that embraces the unknown," IEEE Robotics and Automation Letters, vol. 5, no. 4, pp. 6411–6418, 2020.
[21] S. Macenski, D. Tsai, and M. Feinberg, "Spatio-temporal voxel layer: A view on robot perception for the dynamic world," International Journal of Advanced Robotic Systems, vol. 17, no. 2, 2020.
[22] D. V. Lu, D. Hershberger, and W. D. Smart, "Layered costmaps for context-sensitive navigation," in 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Chicago, IL, USA, 2014, pp. 709–715.
[23] S. Macenski and I. Jambrecic, "SLAM toolbox: SLAM for the dynamic world," Journal of Open Source Software, vol. 6, no. 61, p. 2783, 2021.
[24] Open Robotics, "Creating and using plugins (c++) — ROS 2 documentation," https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Pluginlib.html, 2024, accessed 2026-07-02.